Project 2 Group Report: Mobile Money Investigations with PostgreSQL and Neo4j

Oct 24, 2026 · @hello claude connect to my powerbi project called cenfri pi on

Adventist University of Central Africa · Advanced Big Data Analytics · Project 2

Investigating suspicious mobile-money activity with PostgreSQL and Neo4j on 20,003,300 transfers (1 to 30 September 2026, CAT)

No.	Full name	Student ID
1	Amie Marie Flora Dushimumukiza	101405
2	Kwizera Rene	101379
3	Grace Teta	101378

Lecturer: Temitope Oguntade. The member names are copied from Assignment I; confirm they are the Project 2 group. Fill the figure placeholders (marked **[Figure]**) with screenshots that show the taskbar and clock.

1 Build the environment and capture the data

1.1 Confirm the environment

All work ran on one Windows laptop with 15.87 GiB RAM. PostgreSQL holds the database `mobile_money` on 127.0.0.1:5432, and Neo4j (Neo4j Desktop DBMS, default `neo4j` database) listens on bolt://127.0.0.1:7687. Python runs in a virtual environment with `psycopg` 3.3.6 and the `neo4j` driver 6.4.1. The PostgreSQL session time zone is set to Africa/Kigali so every date boundary is CAT.

Item	Value
Operating system	Windows (taskbar clock shows CAT)
RAM	15.87 GiB total
Neo4j version and edition	2026.05.0; folder name says enterprise, so state the edition used **[confirm]**

Item	Value
Java used for neo4j-admin	Java 21 (Java 17 failed with UnsupportedClassVersionError)
Neo4j import resources	max heap 3.556 GiB, page cache 1.992 GiB, 8 worker threads
PostgreSQL version, `shared_buffers`, `work_mem`	**[fill from psql: SHOW ...]**
CPU model and cores	**[fill]**
Python and driver versions	psycopg 3.3.6, neo4j 6.4.1

[Figure 1.1-1] Hardware and software versions. **[Figure 1.1-2]** PostgreSQL and Neo4j settings and running services.

1.2 Capture the data

The unchanged generator `mobile_money_generator.py` is imported by `capture.py`, which prints `g.dataset_info()` and `g.FIELDS`, previews ten rows per stream and streams each of the five generators to CSV with `csv.DictWriter`, never holding a stream in memory. The audit totals for transfers (row count, earliest and latest `occurred_at`, sum of `amount_rwf`) were saved and reused to check both databases.

File	Rows	Size (bytes)
customers.csv	228,000	3,353,914
accounts.csv	248,000	4,964,712
devices.csv	228,600	3,325,917
account_devices.csv	285,600	5,998,888
transfers.csv	20,003,300	760,194,544

Capture seconds, the audit sum, minimum and maximum time: **[fill from the capture log and audit JSON]**. **[Figure 1.2-1]** Preview and audit output.

2 PostgreSQL

2.1 Schema, load and indexes

Five tables hold the five streams, every column `NOT NULL`. IDs, amounts and observations are `INTEGER`; `transfer_id` and `occurred_at` are `BIGINT` (Unix seconds); observation days are `SMALLINT`; categories are `TEXT`. Primary keys: one per table, with `(account_id, device_id)` for `account_devices`. Foreign keys cover account ownership, device use and both transfer endpoints. Checks: `amount_rwf > 0`, `observations > 0`, `0 <= first_day <= last_day <= 29`.

Data was loaded with `\copy` in dependency order (customers, devices, accounts, account_devices, transfers), with keys and indexes added after loading transfers.

Index	Purpose
`accounts(customer_id, account_id)`	owner lookups and Q4
`account_devices(device_id, account_id)`	shared-device lookups (Q3, Q4)
`transfers(sender_id, occurred_at)` INCLUDE `receiver_id, amount_rwf, transfer_id`	outgoing activity (Q1, Q5, Q6, Q7)
`transfers(receiver_id, occurred_at)` INCLUDE `sender_id, amount_rwf, transfer_id`	incoming activity
BRIN on `transfers(occurred_at)`	time-range scans (Q2)

`ANALYZE` was run after loading. **[Figure 2.1-1]** Table definitions and constraints. **[Figure 2.1-2]** Indexes and sizes (1 GB = 1,000,000,000 bytes).

2.2 Verify the load

Load seconds, index and constraint seconds, database size and table sizes: **[fill from psql and your load log]**. Row counts must equal 228,000 / 248,000 / 228,600 / 285,600 / 20,003,300; account roles must be 240,000 person, 6,000 merchant and 2,000 agent; the transfer count, sum, minimum and maximum time must equal the capture audit. **[Figure 2.2-1]** `verify_counts.py` PostgreSQL output.

3 Neo4j

3.1 Graph model

Nodes are `Customer(customer_id, region)`, `Account(account_id, role)` and `Device(device_id, access_type)`. Relationships are `OWNS` (customer to account, from `accounts.customer_id`), `USED_DEVICE` (account to device, with `first_day`, `last_day`, `observations`) and `TRANSFER` (sender to receiver, with `transfer_id`, `occurred_at`, `amount_rwf`). Each transfer is its own relationship; nothing is collapsed. **[Figure 3.1-1]** Graph schema drawing with every source field mapped.

3.2 Import and verification

`prepare_import.py` wrote temporary typed copies (`customer_id:ID(Customer)`, `account_id:ID(Account)`, `device_id:ID(Device)`, accounts header `account_id:ID(Account),:IGNORE,role:string`) and derived `owns.csv` from accounts in customer_id, account_id order, leaving the source CSVs unchanged. Neo4j was stopped and the import ran with `neo4j-admin database import full neo4j --id-type=integer`, using Java 21.

Measure	Result
Import time reported by the tool	50.4 s (command stopwatch including start-up: 133.7 s)
Nodes	704,600 (228,000 customers, 248,000 accounts, 228,600 devices)
Relationships	20,536,900 (248,000 OWNS, 285,600 USED_DEVICE, 20,003,300 TRANSFER)
Store size	2.527 GB
Transaction logs	about 540 bytes
Type normalization	`int` became `long` for amount, first_day, last_day, observations

After the import, unique constraints were confirmed on the three business IDs and a range index on `TRANSFER.occurred_at`. Checks passed: roles 240,000 / 6,000 / 2,000, no account without exactly one owner, 20,003,300 distinct `transfer_id`. The temporary

import copies were deleted afterwards **[confirm]**. **[Figure 3.2-1]** Import completion. **[Figure 3.2-2]** `SHOW CONSTRAINTS` and `SHOW INDEXES`. **[Figure 3.2-3]** Count checks.

3.3 Trace one account in both models

Account 998 is traced from its owner customer through a device to an outgoing transfer, once with SQL JOINs over `accounts`, `customers`, `account_devices` and `transfers`, once with a Cypher `MATCH` over `OWNS`, `USED_DEVICE` and `TRANSFER`. Each foreign key becomes a relationship. Collapsing repeated payments between two accounts into one edge would lose the transfer ID, timestamps and amounts, which Q1, Q5 and Q6 depend on. **[Figure 3.3-1]** SQL trace. **[Figure 3.3-2]** Cypher trace.

4 Questions Q1 to Q3

The review accounts and hour windows are the six sets A to F. A window starts at `START + (day-1)*86400 + 9*3600 + 45*60` with `START = 1788213600` (1 September 2026 00:00 CAT) and lasts 3,600 seconds; it is computed in Python and passed as parameters to both engines.

Set	Review account	September day	Q7 destination
A	998	6	24755
B	147228	13	186823
C	206978	8	6573
D	237998	6	21755
E	144228	13	183823
F	203978	8	3573

4.1 Q1: account activity

Outgoing transfer count and total RWF in the hour, over all receiver roles. SQL uses `count(*)` with `COALESCE(sum(...), 0)`; Cypher uses `OPTIONAL MATCH` so an empty hour still returns zeros. The total is outgoing value only, not a balance: incoming transfers, fees and reversals are not in the data. Set A returns 1 transfer of 200,000 RWF. **[Table: Q1 results for A to F, from `all_answers.txt`]** **[Figure 4.1-1]** Query pair and output.

4.2 Q2: daily activity

Transfers grouped by CAT day: SQL uses `to_timestamp(...) AT TIME ZONE 'Africa/Kigali'`, Cypher uses `datetime({epochSeconds: ..., timezone: 'Africa/Kigali'})`. Both return 30 rows. **[Chart: transfers and RWF per day, units labelled]** The daily sums must equal the capture totals. **[Figure 4.2-1]** Output.

4.3 Q3: shared devices

Accounts using the same device with overlapping observation days, excluding the review account: overlap start is the later `first_day`, overlap end the earlier `last_day`, and a pair counts when start <= end. Both handset and public-terminal devices are included. Set A returns 4 rows: 220998 on device 998 and 8917, 16836, 24755 on device 228001, all days 0 to 29. **[Table: Q3 results for A to F]** A shared device is observed use, not proof of ownership or wrongdoing. **[Figure 4.3-1]** Query pair and output.

5 Questions Q4 to Q7

5.1 Q4: ownership and device network

Accounts reachable from the review account through `OWNS` or `USED_DEVICE` links in either direction within 4, 6, 8 and 10 links, grouped by owner region (24 cases). SQL carries node kind with every ID, because Customer 7 is not Account 7, and uses `UNION` in the recursive step so repeated states disappear and the walk ends. Cypher uses an undirected bounded pattern and `DISTINCT` accounts before grouping. Each reachable account is counted once, and the review account itself is excluded. Set A returns 2 accounts at 4 links and 5 at 6, 8 and 10 links, because its connected component is small. **[Predictions for depths 8 and 10, written before the run] [Table and chart: accounts per region by depth for B and E] [Figure 5.1-1]**

5.2 Q5: rapid onward payments

Three directed transfers through four distinct accounts, the first made by the review account inside its hour; each next transfer is strictly later and at most 600 seconds after the previous one, and each amount is between 90% and 105% of the previous (`90*prev <= 100*next <= 105*prev`, with BIGINT arithmetic in SQL). Set A returns one chain (transfers 20000001 to 20000003, final receiver 24755). **[Table for A to F, with CAT times]** The pattern is a prompt for review: a family splitting a remittance could produce the same shape. **[Figure 5.2-1]**

5.3 Q6: circular payments

3 to 8 transfers starting and ending at the review account, all inside the hour, strictly increasing in time, spanning at most 1,800 seconds, with the same amount rule between consecutive transfers; the walk stops at the first return and other accounts may repeat. Set A returns one cycle of length 4: 998 to 8917 (200,000 RWF), 8917 to 16836 (196,000), 16836 to 24755 (192,080), 24755 to 998 (188,238), 40 seconds apart. **[Table for A to F] [Figure 5.3-1]**

5.4 Q7: shortest transfer connection

Fewest directed transfers from the review account to the destination with every account on the route a `person`, bounds 4 and 6 (12 cases). The person restriction is applied during the search in both engines. One path per successful case was retrieved and checked in PostgreSQL for direction, existence of every transfer, roles and length; all six passed.

Set	Path (accounts)	Transfers
A	998, 8917, 16836, 24755	3
B	147228, 148019, 147310, 148101, 178904, 186823	5
C	206978, 214897, 222816, 230735, 238654, 6573	5
D	237998, 5917, 13836, 21755	3
E	144228, 152147, 160066, 167985, 175904, 183823	5
F	203978, 211897, 219816, 227735, 235654, 3573	5

Sets B, C, E and F need five transfers, so they have no route within bound 4 but a route at bound 6: no connection within four transfers does not mean no connection. Equal-length paths may differ between engines, so lengths are compared exactly and each path is validated on its own. **[Figure 5.4-1]** Path validation output.

6 Correctness and performance

6.1 Compare the engines

`run_all.py` runs all 61 cases (Q1 6, Q2 1, Q3 6, Q4 24, Q5 6, Q6 6, Q7 12) on both engines with the same parameters and compares integer-normalised, sorted rows; Q7 lengths are compared exactly and paths validated separately. Result: 61 cases, 0 mismatches, 0 errors. An earlier run also reported 0 mismatches but was invalid: the review-account table still held placeholder zeros, so both engines returned empty results. After the real sets were filled in, Q3 and Q4 returned rows; the empty cases (if any) are listed here: **[list from `summary.csv`]**. Independent checks from the CSV files only: the Q3 overlap for account 998 (220998, 8917, 16836, 24755) and the cycle formed by transfers 20000001 to 20000004. **[Figure 6.1-1]** Status table. **[Figure 6.1-2]** Independent check.

6.2 Time the queries

21 cases per engine: Q1, Q3, Q5 and Q6 on B and E, Q2, Q4 on B and E at four depths, Q7 on B and E at both bounds. One engine runs at a time with fixed memory settings, a persistent connection per engine, 3 untimed warm-ups (180 s limit) then 5 measured runs (60 s limit), timed with `perf_counter` through the last row. Timeouts and errors are logged separately and never counted as timings.

Case	PostgreSQL median / min / max (ms)	Neo4j median / min / max (ms)	PostgreSQL / Neo4j median
[fill from `timing_raw_pg.csv` and `timing_raw_neo.csv`]			

Plans collected outside the timer for Q2, Q4 B depth 10 and Q7 B bound 6 (`EXPLAIN (ANALYZE, BUFFERS)` and `PROFILE`). One processing difference to explain: **[fill after reading the plans]**. **[Figure 6.2-1]** Timing run. **[Figure 6.2-2]** Plans.

6.3 Improve the SQL

Plan evidence of wasted work: **[fill]**. Change tried: **[deduplicate account-depth states, or search from both ends, or stop at the first proven minimum]**. Both versions are kept; all 12 Q7 cases are rechecked against Neo4j and B and E retimed at both bounds. Version

carried forward (chosen on bound-6 medians): **[fill]**. An honest unsuccessful attempt is reported as such.

7 Concurrency

7.1 Design and results

Workloads: Q4 depth 10 and Q7 length-only bound 6 (with the SQL version chosen in 6.3), cycling sets A, B, C. Clients: 1, 4, 8 and 16 on each engine, 16 combinations, each repeated 3 times. Each client is a separate process with one persistent connection and warms up A-B-C once; after a `Barrier` all clients share a start time and a deadline 15 seconds later, keep starting cycles until the deadline and finish any cycle already started. Every request is logged and checked for correctness after timing. Throughput is correct completions divided by the time from barrier release to the last cycle finishing; latency uses median and p95 (position `ceil(0.95*n)`).

Engine	Workload	Clients	Throughput (cycles/s, median of 3)	Median latency (ms)	p95 (ms)	Errors / timeouts
[fill from the concurrency CSV]						

[Chart: throughput against client count] [Chart: p95 against client count]

7.2 Interpret

Compare 1 against 16 clients and explain any saturation with CPU, RAM and plan evidence; keep throughput (work per second) distinct from the response time of one request; state whether any graph advantage survives the SQL improvement of 6.3, and limit the claim to this machine, data size and workload. **[Write after the results]**

8 Decisions

8.1 Prioritise the review

One table with three rows (A/D, B/E, C/F) using Q3 to Q7, then the account to review first or a justification for equal priority.

Pair	Q3 shared devices	Q4 reach	Q5 chain	Q6 cycle	Q7 route	Priority and reason
A / D	**[fill]**	**[fill]**	**[fill]**	**[fill]**	3 transfers (A, D)	**[fill]**
B / E	**[fill]**	**[fill]**	**[fill]**	**[fill]**	5 transfers	**[fill]**
C / F	**[fill]**	**[fill]**	**[fill]**	**[fill]**	5 transfers	**[fill]**

Discuss one pair in detail with account IDs, amounts, CAT times and device evidence; include one chain or cycle diagram and one ownership-device diagram. Give an ordinary explanation for one suspicious pattern, name the missing evidence and how it could change the judgement. Do not declare fraud.

8.2 Recommendation (500 words at most)

Cover reporting (Q1, Q2), traversal (Q4 to Q7) and concurrency, plus load time and storage (PostgreSQL load and index seconds and database size; Neo4j import 50.4 s and store 2.527 GB). Include evidence against the preferred option, separate model benefits from implementation effects, propose a freshness target if operational data is copied into a graph, and state two limits. **[Write after sections 6 and 7]**

8.3 Individual notes

Each member writes 150 to 200 words on a different measured finding, its explanation and a check that could overturn it, with their name and ID. List contributions and any LLM or external help, and how each was verified.

Member	Student ID	Finding discussed
Amie Marie Flora Dushimumukiza	101405	**[fill]**
Kwizera Rene	101379	**[fill]**
Grace Teta	101378	**[fill]**

9 Submit the report and code

9.1 Organise the report

This report is one PDF per group, in milestone order, with schemas, summary tables, charts, observations, judgements and the recommendation. Every required screenshot is embedded beside its subsection with the command or action, the result, the taskbar and the clock; later subsections that only interpret reuse the earlier evidence.

9.2 Make the work reproducible

Generator unchanged. `mobile_money_generator.py` is included unchanged. Verification: `certutil -hashfile mobile_money_generator.py SHA256` gives **[paste hash]**, identical to the issued copy.

Repository and ZIP. GitHub repository: https://github.com/KWIZERARENE/Mobile-Money-Investigations-with-PostgreSQL-and-Neo4j. A ZIP with the same final files is submitted. The CSV files and database stores are not included; the README explains how to regenerate and load them. Passwords in files are placeholders.

Item	Files
Generator (unchanged)	`mobile_money_generator.py`
Capture and import	`capture.py`, `prepare_import.py`
Database definitions	PostgreSQL DDL, indexes, Neo4j constraints and index
Queries	`queries.py`, `cases.py`, `timeutil.py`, `db.py`
Checks	`run_all.py`, `save_all.py`, `verify_counts.py`, `independent_check.py`, `q7_paths.py`
Benchmarks and plans	`bench.py`, `plans.py`, concurrency script, raw result CSVs, plan files
Documentation	`README.md` with execution order; this report

README execution order.

```
1. python capture.py                stream the five generators to CSV and
save audit totals
2. psql ... -f schema.sql           create the PostgreSQL tables
3. \copy ... then indexes, ANALYZE  load PostgreSQL
4. python prepare_import.py         typed temporary CSVs for Neo4j
5. neo4j-admin database import full stop Neo4j first, then import
6. constraints and TRANSFER index   run in Neo4j, wait until ONLINE
7. python verify_counts.py          counts and audit totals in both
databases
8. python run_all.py                61-case comparison
9. python bench.py pg / neo         timing, one database running at a time
10. python plans.py pg / neo        EXPLAIN and PROFILE
11. concurrency script              load test
```

Final check: the 61-case comparison is rerun and screenshotted, then both database services are stopped and the stopped status is screenshotted. The stores are kept. **[Figure 9.2-1]** Final comparison run. **[Figure 9.2-2]** Services stopped.

9.3 Document contributions

Member	Student ID	Individual contributions
Amie Marie Flora Dushimumukiza	101405	**[fill]**
Kwizera Rene	101379	**[fill]**
Grace Teta	101378	**[fill]**